

Réseau & Linux pour le DevOps

Cours complet — théorie, commandes, exercices corrigés

Formateur : Haithem Mihoubi

Formation DevOps • 14 modules • 2026

Table des matières

Module 00 — Introduction au DevOps	3
Module 01 — Linux : Fondamentaux	5
Module 02 — Linux : Utilisateurs, Groupes & Permissions	9
Module 03 — Linux : Processus, Services & Tâches planifiées	13
Module 04 — Linux : Gestion des paquets & Stockage	17
Module 05 — Bash Scripting	21
Module 06 — Réseau : Fondamentaux & Modèles	27
Module 07 — Réseau : Adressage IP & Sous-réseaux	31
Module 07b — L'adressage IP expliqué pas à pas (+ exercices corrigés)	35
Module 08 — Réseau : Protocoles & Services	44
Module 09 — Réseau : Diagnostic & Outils Linux	49
Module 10 — Sécurité & Pare-feu	53
Module 11 — Mise en pratique DevOps	57
Module 12 — Travaux Pratiques & Exercices corrigés	62

Module 00 — Introduction au DevOps

🎯 **Objectif** : comprendre ce qu'est le DevOps et pourquoi Linux et le réseau en sont les fondations.

1. Qu'est-ce que le DevOps ?

Le **DevOps** est une culture et un ensemble de pratiques qui rapprochent les équipes de **développement** (Dev) et d'**exploitation/opérations** (Ops). L'objectif est de livrer des logiciels **plus vite, plus souvent** et **plus fiablement**.

Avant le DevOps, on avait un « mur de confusion » : - Les **Devs** écrivaient le code et le « jetaient par-dessus le mur ». - Les **Ops** devaient le déployer et le maintenir, souvent sans contexte.

Le DevOps casse ce mur grâce à l'automatisation, la collaboration et le partage de responsabilités.

2. Les piliers du DevOps (modèle CALMS)

Pilier	Signification
Culture	Collaboration, responsabilité partagée, pas de blâme
Automation	Automatiser builds, tests, déploiements, infrastructure
Lean	Réduire le gaspillage, livrer de petites itérations
Measurement	Mesurer tout (perf, erreurs, temps de déploiement)
Sharing	Partager connaissances, outils, retours d'expérience

3. Le cycle de vie DevOps (l'infini ∞)

PLAN → CODE → BUILD → TEST → RELEASE → DEPLOY → OPERATE → MONITOR → (retour à PLAN)

- **Plan** : Jira, backlog, user stories (lien avec Scrum)
- **Code** : Git, revue de code
- **Build** : Maven, Gradle, npm
- **Test** : tests unitaires, intégration
- **Release/Deploy** : CI/CD (GitLab CI, Jenkins, GitHub Actions)
- **Operate** : Kubernetes, Docker, serveurs Linux
- **Monitor** : Prometheus, Grafana, logs

4. Pourquoi Linux est incontournable en DevOps

- ~90 % des serveurs en production tournent sous Linux.
- Docker, Kubernetes, la majorité des outils cloud sont nés sur Linux.
- Les serveurs web (Nginx, Apache), bases de données et CI/CD runners tournent sous Linux.
- Tout s'automatise en ligne de commande → maîtriser le shell est vital.

5. Pourquoi le réseau est incontournable en DevOps

Un DevOps passe son temps à connecter des choses : - Une application qui parle à une base de données - Un load balancer qui répartit le trafic - Des conteneurs qui communiquent dans un cluster - Des appels d'API entre microservices

Sans comprendre **IP, ports, DNS, TCP, HTTP, pare-feu**, impossible de diagnostiquer un « ça ne marche pas ».

6. Outils que vous croiserez

Catégorie	Exemples
OS	Ubuntu, Debian, CentOS, Rocky Linux, Alpine
Conteneurs	Docker, Podman, containerd
Orchestration	Kubernetes, Docker Swarm
CI/CD	Jenkins, GitLab CI, GitHub Actions
IaC	Terraform, Ansible
Monitoring	Prometheus, Grafana, ELK
Cloud	AWS, Azure, GCP

7. Quiz de fin de module

1. Que signifie l'acronyme CALMS ?
2. Citez 3 raisons pour lesquelles un DevOps doit maîtriser Linux.
3. Dans le cycle DevOps, quelle phase suit le « Deploy » ?
4. Pourquoi parle-t-on de « mur de confusion » avant le DevOps ?

✓ Une fois ce module compris, passez au [Module 01 — Linux Fondamentaux](#).

Module 01 — Linux : Fondamentaux

 **Objectif** : être à l'aise dans un terminal Linux, naviguer, manipuler fichiers et texte.

1. Architecture de Linux

```
+-----+
| Applications (bash, nginx...) |
+-----+
| Shell (interface utilisateur) |
+-----+
| Noyau (Kernel)                | <- gère CPU, mémoire, périphériques
+-----+
| Matériel (Hardware)           |
+-----+
```

- **Noyau (kernel)** : cœur du système, gère les ressources.
- **Shell** : interpréteur de commandes (le plus courant : `bash`, aussi `zsh`, `sh`).
- **Distribution** : noyau + outils + gestionnaire de paquets (Ubuntu, Debian, CentOS...).

2. Le terminal et le shell

```
whoami      # affiche l'utilisateur courant
hostname    # nom de la machine
date        # date et heure
uname -a    # infos sur le noyau et l'OS
echo "Bonjour" # affiche du texte
pwd          # répertoire courant (Print Working Directory)
```

 La structure d'une commande : `commande [options] [arguments]` Exemple : `ls -l /home`
→ commande `ls`, option `-l`, argument `/home`.

3. Arborescence du système de fichiers (FHS)

Sous Linux, **tout est fichier** et tout part de la racine `/`.

Dossier	Rôle
<code>/</code>	Racine de tout le système
<code>/home</code>	Dossiers personnels des utilisateurs
<code>/root</code>	Dossier personnel de root
<code>/etc</code>	Fichiers de configuration
<code>/var</code>	Données variables (logs dans <code>/var/log</code>)
<code>/tmp</code>	Fichiers temporaires
<code>/usr</code>	Programmes et bibliothèques
<code>/bin</code> , <code>/sbin</code>	Commandes système essentielles
<code>/opt</code>	Logiciels tiers
<code>/dev</code>	Périphériques (disques, etc.)
<code>/proc</code> , <code>/sys</code>	Infos noyau en temps réel (virtuels)
<code>/mnt</code> , <code>/media</code>	Points de montage

4. Navigation dans les fichiers

```

pwd          # où suis-je ?
ls           # lister le contenu
ls -l        # format long (droits, taille, date)
ls -la       # inclut les fichiers cachés (commençant par .)
ls -lh       # tailles lisibles (K, M, G)
cd /var/log  # aller dans un dossier (chemin absolu)
cd ..        # remonter d'un niveau
cd ~         # aller dans son home
cd -         # revenir au dossier précédent
tree         # affiche l'arborescence (à installer)

```

📌 **Chemin absolu** vs **relatif** : - Absolu : commence par `/` → `/home/haithem/docs` -
 Relatif : par rapport au dossier courant → `docs/notes.txt` , `../config`

5. Manipuler fichiers et dossiers

```

mkdir projet          # créer un dossier
mkdir -p a/b/c        # créer une arborescence complète
touch fichier.txt      # créer un fichier vide
cp source.txt dest.txt # copier un fichier
cp -r dossier1 dossier2 # copier un dossier (récursif)
mv ancien.txt nouveau.txt # renommer ou déplacer
rm fichier.txt         # supprimer un fichier
rm -r dossier          # supprimer un dossier
rm -rf dossier         # forcer la suppression ⚠ DANGEREUX

```

⚠ `rm -rf /` peut détruire tout le système. Vérifiez toujours deux fois.

6. Lire et afficher le contenu

```
cat fichier.txt          # affiche tout le fichier
less fichier.txt         # lecture page par page (q pour quitter)
head -n 20 fichier      # 20 premières lignes
tail -n 20 fichier       # 20 dernières lignes
tail -f /var/log/syslog  # suit le fichier en temps réel (très utile DevOps !)
wc -l fichier            # compte le nombre de lignes
```

💡 `tail -f` est essentiel pour surveiller les logs d'une application en direct.

7. Rechercher des fichiers et du texte

```
find /etc -name "*.conf"      # trouver des fichiers par nom
find /var -type f -size +100M # fichiers de plus de 100 Mo
find . -mtime -1              # modifiés il y a moins d'1 jour

grep "erreur" fichier.log     # chercher un mot dans un fichier
grep -i "error" app.log       # insensible à la casse
grep -r "TODO" .              # récursif dans tous les fichiers
grep -n "404" access.log      # affiche les numéros de ligne
grep -v "INFO" app.log        # inverse : lignes SANS "INFO"
```

8. Les redirections et les pipes

C'est le **superpouvoir** du shell : combiner des commandes.

```
echo "test" > fichier.txt    # écrit dans fichier (écrase)
echo "ligne 2" >> fichier.txt # ajoute à la fin (append)
commande 2> erreurs.log      # redirige les erreurs (stderr)
commande > sortie.log 2>&1    # redirige sortie + erreurs

# Le pipe | envoie la sortie d'une commande vers une autre
cat access.log | grep "404" | wc -l    # compte les erreurs 404
ps aux | grep nginx                  # filtre les processus nginx
ls -l /etc | sort | head              # trie et prend le début
```

📊 **Flux standards** : - `stdin` (0) : entrée - `stdout` (1) : sortie normale - `stderr` (2) : sortie d'erreur

9. Éditer des fichiers (nano & vim)

```
nano fichier.txt    # éditeur simple (Ctrl+O sauver, Ctrl+X quitter)
vim fichier.txt      # éditeur puissant
```

Bases de Vim (incontournable car présent partout) : - `i` : mode insertion - `Échap` : revenir en mode normal - `:w` : sauvegarder - `:q` : quitter - `:wq` : sauvegarder et quitter - `:q!` : quitter sans sauver - `/mot` : rechercher

10. Aide et documentation

```
man ls          # manuel d'une commande
ls --help       # aide rapide
which python3   # où se trouve un exécutable
type cd         # type d'une commande
apropos network # chercher des commandes par mot-clé
```

Exercices

1. Créez l'arborescence `~/devops/projet1/logs` en une seule commande.
2. Créez un fichier `notes.txt`, écrivez-y 3 lignes avec `echo` et `>>`.
3. Affichez uniquement les lignes contenant le mot « erreur » d'un fichier log.
4. Comptez combien de fichiers `.conf` existent dans `/etc` (et sous-dossiers).
5. Copiez `/etc/hostname` dans votre home, renommez-le `mon-host.txt`.
6. Utilisez un pipe pour afficher les 5 plus gros fichiers de `/var/log`.

✓ Passez au [Module 02 — Utilisateurs & Permissions](#).

Module 02 — Linux : Utilisateurs, Groupes & Permissions

 **Objectif** : maîtriser la sécurité d'accès aux fichiers, essentielle en production.

1. Le modèle multi-utilisateurs

Linux est nativement multi-utilisateurs. Chaque utilisateur a : - un **UID** (User ID) unique - un ou plusieurs **groupes** (GID) - un dossier personnel (`/home/utilisateur`) - un shell par défaut

Le super-utilisateur **root** (UID 0) a tous les droits.

```
id                # affiche UID, GID et groupes de l'utilisateur courant
whoami            # nom de l'utilisateur
who              # qui est connecté
cat /etc/passwd   # liste des utilisateurs
cat /etc/group    # liste des groupes
cat /etc/shadow   # mots de passe (chiffrés, root only)
```

Format d'une ligne `/etc/passwd` :

```
haithem:x:1000:1000:Haithem:/home/haithem:/bin/bash
|      |  |  |      |      |      |
nom    pwd UID  GID  commentaire home    shell
```

2. Gérer les utilisateurs

```
sudo useradd -m -s /bin/bash alice # créer un user avec home et bash
sudo passwd alice                  # définir son mot de passe
sudo usermod -aG docker alice      # ajouter alice au groupe docker
sudo userdel -r alice              # supprimer alice et son home
sudo adduser bob                   # alternative interactive (Debian/Ubuntu)
```

 **-aG** = **append** au **Groupe**. Oublier le **-a** retire les autres groupes !

3. Gérer les groupes

```
sudo groupadd developpeurs    # créer un groupe
sudo gpasswd -a alice developpeurs # ajouter alice au groupe
groups alice                  # voir les groupes d'alice
sudo groupdel developpeurs    # supprimer le groupe
```

4. sudo et l'élévation de privilèges

`sudo` permet d'exécuter une commande en tant que root (ou autre user).

```
sudo apt update    # exécute en root
sudo -i            # ouvre un shell root
sudo -u alice commande # exécute en tant qu'alice
visudo            # éditer les droits sudo en sécurité
```

⚠ **Bonne pratique DevOps** : on ne se connecte JAMAIS directement en root sur un serveur. On utilise un compte normal + `sudo`. Cela permet la traçabilité (qui a fait quoi).

5. Comprendre les permissions

```
ls -l fichier.txt
# -rw-r--r-- 1 haithem developpeurs 1024 juin 5 10:00 fichier.txt
```

Décortiquons `-rw-r--r--` :

-	rw-	r--	r--
type	proprio	groupe	autres

- **1er caractère** : type (`-` fichier, `d` dossier, `l` lien)
- **3 blocs de 3** : droits du **propriétaire**, du **groupe**, des **autres**
- `r` = read (lecture, valeur 4)
- `w` = write (écriture, valeur 2)
- `x` = execute (exécution, valeur 1)

6. Notation octale (numérique)

Chaque bloc = somme des valeurs :

Droits	Calcul	Octal
<code>rwX</code>	4+2+1	7
<code>rw-</code>	4+2	6
<code>r-X</code>	4+1	5
<code>r--</code>	4	4
<code>---</code>	0	0

Exemples courants : - `755` = `rwXr-Xr-X` (scripts, dossiers) - `644` = `rw-r--r--` (fichiers de config) - `600` = `rw-----` (clés privées, secrets) - `777` = `rwXrwXrwX` ⚠ à éviter (tout le monde peut tout faire)

7. Modifier les permissions

```

chmod 755 script.sh      # notation octale
chmod +x script.sh       # rendre exécutable
chmod -x script.sh       # retirer l'exécution
chmod u+w fichier        # ajouter écriture au propriétaire (user)
chmod g-r fichier        # retirer lecture au groupe
chmod o=r fichier        # autres = lecture seule
chmod -R 755 dossier/    # récursif sur tout un dossier

```

Notation symbolique : `u` =user, `g` =group, `o` =others, `a` =all ; `+` / `-` / `=` .

8. Changer le propriétaire

```

sudo chown alice fichier.txt      # change le propriétaire
sudo chown alice:devs fichier.txt # propriétaire ET groupe
sudo chown -R alice:devs /app     # récursif
sudo chgrp devs fichier.txt       # change uniquement le groupe

```

9. Cas pratiques DevOps

```

# Sécuriser une clé SSH privée
chmod 600 ~/.ssh/id_rsa

# Rendre un script de déploiement exécutable
chmod +x deploy.sh

# Donner à l'app web les droits sur son dossier
sudo chown -R www-data:www-data /var/www/monsite

# Permissions correctes pour un dossier web
sudo find /var/www -type d -exec chmod 755 {} \;
sudo find /var/www -type f -exec chmod 644 {} \;

```

10. Permissions spéciales (notions avancées)

- **SUID** (**s** sur user) : le programme s'exécute avec les droits du propriétaire.
- **SGID** (**s** sur group) : héritage du groupe.
- **Sticky bit** (**t**) : sur `/tmp`, seul le propriétaire peut supprimer ses fichiers.

```
chmod u+s programme    # SUID
chmod g+s dossier       # SGID
chmod +t /partage       # sticky bit
```

Exercices

1. Créez un utilisateur `deploy` avec un home et le shell bash.
2. Créez un groupe `web` et ajoutez-y `deploy`.
3. Créez un fichier et donnez-lui les permissions `640` en octal. Vérifiez avec `ls -l`.
4. Quel est l'octal de `rwxr-x---` ?
5. Sécurisez un fichier `secret.key` pour que seul le propriétaire puisse le lire/écrire.
6. Changez le propriétaire d'un dossier `/app` vers `deploy:web` récursivement.

✓ Passez au [Module 03 — Processus & Services](#).

Module 03 — Linux : Processus, Services & Tâches planifiées

 **Objectif** : surveiller et contrôler ce qui tourne sur un serveur, gérer les services et automatiser des tâches.

1. Qu'est-ce qu'un processus ?

Un **processus** est un programme en cours d'exécution. Chacun a : - un **PID** (Process ID) unique - un **PPID** (PID du parent) - un propriétaire - un état (running, sleeping, zombie...)

2. Lister les processus

```
ps aux          # tous les processus du système
ps aux | grep nginx # filtrer un processus précis
ps -ef         # format alternatif
pstree         # arborescence des processus
top            # moniteur temps réel (q pour quitter)
htop           # version améliorée et colorée (à installer)
```

Colonnes de `ps aux` : USER, PID, %CPU, %MEM, VSZ, RSS, STAT, START, TIME, COMMAND.

3. Surveiller les ressources

```
top            # CPU, mémoire, processus en direct
htop           # interactif (F9 pour tuer, F6 pour trier)
free -h       # mémoire RAM et swap (format lisible)
uptime        # charge système (load average)
vmstat 1      # statistiques système chaque seconde
df -h         # espace disque par partition
du -sh /var/* # taille des dossiers
iostat        # statistiques d'E/S disque
```

 **Load average** : 3 chiffres = charge moyenne sur 1, 5 et 15 min. Sur une machine 4 cœurs, une charge > 4 indique une saturation.

4. Tuer / signaler un processus

```
kill 1234          # envoie SIGTERM (arrêt propre) au PID 1234
kill -9 1234       # SIGKILL (arrêt forcé, dernier recours)
kill -15 1234      # SIGTERM explicite
killall nginx      # tue tous les processus nommés nginx
pkill -f "python app" # tue par motif de ligne de commande
```

Signaux courants : - **SIGTERM** (15) : demande polie d'arrêt - **SIGKILL** (9) : tue immédiatement (ne peut être ignoré) - **SIGHUP** (1) : souvent utilisé pour recharger la config

5. Jobs et arrière-plan

```
commande &          # lance en arrière-plan
Ctrl + Z            # suspend le processus courant
bg                  # reprend en arrière-plan
fg                  # ramène au premier plan
jobs                # liste les jobs
nohup commande &    # continue après déconnexion
```

💡 En production, on préfère gérer les services longue durée via **systemd** plutôt que **nohup**.

6. systemd : le gestionnaire de services moderne

systemd est le système d'init de la plupart des distributions modernes. Il gère le démarrage et les **services** (appelés *units*).

```
sudo systemctl status nginx    # état d'un service
sudo systemctl start nginx     # démarrer
sudo systemctl stop nginx      # arrêter
sudo systemctl restart nginx   # redémarrer
sudo systemctl reload nginx    # recharger la config sans couper
sudo systemctl enable nginx    # démarrage auto au boot
sudo systemctl disable nginx   # désactiver le démarrage auto
sudo systemctl is-active nginx # actif ou non
systemctl list-units --type=service # lister les services
```

7. Créer son propre service systemd (très utile DevOps)

Fichier `/etc/systemd/system/monapp.service` :

```
[Unit]
Description=Mon application Node.js
After=network.target

[Service]
User=deploy
WorkingDirectory=/opt/monapp
ExecStart=/usr/bin/node server.js
Restart=always
Environment=NODE_ENV=production

[Install]
WantedBy=multi-user.target
```

Puis :

```
sudo systemctl daemon-reload    # recharge la config systemd
sudo systemctl enable --now monapp
sudo systemctl status monapp
```

8. Consulter les journaux (logs) avec journalctl

```
journalctl                # tous les logs
journalctl -u nginx       # logs d'un service précis
journalctl -u nginx -f    # suivre en temps réel
journalctl --since "1 hour ago" # depuis 1 heure
journalctl --since today   # aujourd'hui
journalctl -p err         # uniquement les erreurs
journalctl -n 50          # les 50 dernières lignes
```

Logs classiques aussi dans `/var/log/` :

```
tail -f /var/log/syslog      # journal système (Debian/Ubuntu)
tail -f /var/log/auth.log    # authentications
tail -f /var/log/nginx/error.log # erreurs nginx
```

9. Planifier des tâches : cron

cron exécute des commandes à intervalles réguliers.

```
crontab -e    # éditer ses tâches planifiées
crontab -l    # lister ses tâches
```

Format d'une ligne cron :

```
* * * * * commande
| | | | |
| | | | +-- jour de la semaine (0-7, 0 et 7 = dimanche)
| | | +---- mois (1-12)
| | +----- jour du mois (1-31)
| +----- heure (0-23)
+----- minute (0-59)
```

Exemples :

```
0 2 * * * /opt/scripts/backup.sh # tous les jours à 2h
*/5 * * * * /opt/scripts/check.sh # toutes les 5 minutes
0 0 * * 0 /opt/scripts/weekly.sh # chaque dimanche à minuit
30 8 1 * * /opt/scripts/monthly.sh # le 1er du mois à 8h30
```

💡 Alternative moderne : les **timers systemd** (plus robustes, journalisés).

10. Variables d'environnement

```
echo $PATH # affiche le PATH
export API_KEY="secret" # définir une variable d'environnement
env # lister toutes les variables
printenv HOME # afficher une variable
```

Pour les rendre persistantes : `~/.bashrc`, `~/.profile` ou `/etc/environment`.

Exercices

1. Trouvez le PID du processus consommant le plus de CPU.
2. Lancez `sleep 300 &` puis ramenez-le au premier plan, puis tuez-le.
3. Affichez l'état du service SSH (`ssh` ou `sshd`).
4. Écrivez un service systemd qui lance un script `/opt/hello.sh`.
5. Programmez une tâche cron qui exécute un script chaque jour à 23h.
6. Affichez en temps réel les logs du service SSH avec `journalctl`.

✅ Passez au **Module 04 — Paquets & Stockage**.

Module 04 — Linux : Gestion des paquets & Stockage

 **Objectif** : installer des logiciels et gérer disques, partitions et systèmes de fichiers.

1. Les gestionnaires de paquets

Chaque famille de distribution a son gestionnaire :

Famille	Gestionnaire	Format
Debian / Ubuntu	apt , dpkg	.deb
RHEL / CentOS / Rocky / Fedora	dnf , yum , rpm	.rpm
Alpine	apk	.apk
Arch	pacman	

2. APT (Debian / Ubuntu)

```
sudo apt update          # met à jour la liste des paquets
sudo apt upgrade         # met à jour les paquets installés
sudo apt install nginx   # installer un paquet
sudo apt install -y git curl # installer sans confirmation
sudo apt remove nginx    # désinstaller (garde la config)
sudo apt purge nginx     # désinstaller + config
sudo apt autoremove      # supprime les dépendances inutiles
apt search docker        # rechercher un paquet
apt show nginx           # infos sur un paquet
dpkg -l                 # liste des paquets installés
dpkg -L nginx           # fichiers installés par un paquet
```

 Toujours faire `apt update` **avant** un `apt install`.

3. DNF / YUM (RHEL / CentOS / Rocky)

```
sudo dnf check-update    # vérifier les mises à jour
sudo dnf update          # tout mettre à jour
sudo dnf install nginx   # installer
sudo dnf remove nginx    # désinstaller
dnf search docker        # rechercher
dnf info nginx           # infos
rpm -qa                 # paquets installés
```

4. APK (Alpine — très utilisé dans Docker)

```
apk update
apk add nginx curl
apk del nginx
apk search git
```

Alpine est très léger → image Docker de base populaire.

5. Installer depuis les sources / binaires

```
# Télécharger un binaire
curl -LO https://example.com/outil
chmod +x outil
sudo mv outil /usr/local/bin/

# Compilation classique (rare en DevOps mais à connaître)
./configure
make
sudo make install
```

6. Comprendre le stockage

```
lsblk          # liste les disques et partitions (arborescence)
fdisk -l       # détails des disques (root)
df -h          # espace utilisé par système de fichiers
df -i          # nombre d'inodes
du -sh /var    # taille d'un dossier
blkid          # UUID et types des partitions
```

Exemple de sortie `lsblk` :

```
NAME  SIZE TYPE MOUNTPOINT
sda    50G disk
├─sda1  1G part /boot
└─sda2 49G part /
```

7. Partitionner et formater

```
sudo fdisk /dev/sdb    # partitionner (interactif)
sudo mkfs.ext4 /dev/sdb1 # formater en ext4
sudo mkfs.xfs /dev/sdb1  # formater en xfs
```

Systèmes de fichiers courants : **ext4** (standard Linux), **xfs** (gros volumes), **btrfs**, **vfat** (compatibilité Windows).

8. Monter et démonter

```
sudo mkdir /mnt/data
sudo mount /dev/sdb1 /mnt/data # monter une partition
sudo umount /mnt/data         # démonter
mount                          # voir les montages actifs
```

Montage **permanent** via `/etc/fstab` :

```
# <device>          <point de montage> <type> <options> <dump> <pass>
UUID=xxxx-xxxx      /mnt/data          ext4   defaults  0       2
```

```
sudo mount -a # teste/applique le fstab
```

⚠ Une erreur dans `/etc/fstab` peut empêcher le serveur de démarrer. Toujours tester avec `mount -a`.

9. LVM (Logical Volume Manager) — notion

LVM ajoute une couche d'abstraction pour redimensionner le stockage à chaud.

Disques physiques (PV) → Groupe de volumes (VG) → Volumes logiques (LV)

```
sudo pvcreate /dev/sdb # créer un volume physique
sudo vgcreate vg_data /dev/sdb # créer un groupe de volumes
sudo lvcreate -L 10G -n lv_app vg_data # créer un volume logique
sudo lvextend -L +5G /dev/vg_data/lv_app # agrandir à chaud
```

10. Surveiller l'espace (cas DevOps fréquent)

Un disque plein casse une appli. Réflexes :

```
df -h # quelle partition est pleine ?
du -sh /var/log/* | sort -h # quel dossier prend de la place ?
journalctl --vacuum-size=200M # nettoyer les vieux logs systemd
docker system prune # nettoyer Docker (images/conteneurs)
```

Exercices

1. Mettez à jour la liste des paquets et installez `htop` et `tree`.
2. Affichez tous les fichiers installés par le paquet `curl`.
3. Listez les disques et partitions de votre machine.
4. Identifiez la partition la plus remplie sur votre système.

5. Trouvez les 3 plus gros dossiers sous `/var`.
6. Expliquez la différence entre `apt remove` et `apt purge`.

✓ Passez au [Module 05 — Bash Scripting](#).

Module 05 — Bash Scripting

 **Objectif** : automatiser des tâches répétitives, base de l'automatisation DevOps.

1. Premier script

Un script bash est un fichier texte de commandes.

```
#!/bin/bash
# mon premier script
echo "Bonjour DevOps !"
```

- La première ligne `#!/bin/bash` s'appelle le **shebang** : elle indique l'interpréteur.
- Rendez le script exécutable et lancez-le :

```
chmod +x script.sh
./script.sh
# ou
bash script.sh
```

2. Variables

```
#!/bin/bash
nom="Haithem"
age=25
echo "Je m'appelle $nom et j'ai $age ans"
echo "Avec accolades : ${nom}_devops"
```

 **Pas d'espace** autour du `=` : `nom="x"` , `nom = "x"` .

Variables spéciales :

```
$0    # nom du script
$1    # premier argument
$2    # deuxième argument
$@    # tous les arguments
$#    # nombre d'arguments
$?    # code de retour de la dernière commande (0 = succès)
$$    # PID du script
```

3. Lire une entrée utilisateur

```
#!/bin/bash
read -p "Quel est votre nom ? " nom
echo "Bonjour $nom"
read -sp "Mot de passe : " mdp # -s = caché
echo
```

4. Substitution de commande

```
date_du_jour=$(date +%Y-%m-%d)
echo "Nous sommes le $date_du_jour"
nb_fichiers=$(ls | wc -l)
echo "Il y a $nb_fichiers fichiers"
```

5. Conditions

```
#!/bin/bash
age=20
if [ $age -ge 18 ]; then
    echo "Majeur"
elif [ $age -ge 13 ]; then
    echo "Adolescent"
else
    echo "Enfant"
fi
```

Opérateurs de comparaison numériques : - `-eq` égal, `-ne` différent - `-gt` >, `-ge` >=, `-lt` <, `-le` <=

Comparaison de chaînes : - `=` égal, `!=` différent, `-z` vide, `-n` non vide

Tests de fichiers : - `-f` fichier existe, `-d` dossier existe - `-e` existe, `-r` lisible, `-w` inscriptible, `-x` exécutable

```
if [ -f /etc/passwd ]; then
    echo "Le fichier existe"
fi

if [ -d /var/log ] && [ -w /var/log ]; then
    echo "Dossier accessible en écriture"
fi
```

💡 `[[ ]]` (double crochet) est plus puissant et recommandé en bash.

6. Boucles

Boucle for :

```
for i in 1 2 3 4 5; do
    echo "Itération $i"
done

for fichier in *.log; do
    echo "Traitement de $fichier"
done

for i in {1..10}; do
    echo "Nombre $i"
done
```

Boucle while :

```
compteur=1
while [ $compteur -le 5 ]; do
    echo "Compteur : $compteur"
    compteur=$((compteur + 1))
done
```

Lire un fichier ligne par ligne :

```
while read ligne; do
    echo "Ligne : $ligne"
done < fichier.txt
```

7. Fonctions

```
#!/bin/bash
saluer() {
    echo "Bonjour $1, bienvenue !"
}

verifier_service() {
    if systemctl is-active --quiet $1; then
        echo "$1 est actif"
        return 0
    else
        echo "$1 est arrêté"
        return 1
    fi
}

saluer "Haithem"
verifier_service "nginx"
```

8. Opérations arithmétiques

```
a=10
b=3
echo $((a + b))    # 13
echo $((a - b))    # 7
echo $((a * b))    # 30
echo $((a / b))    # 3 (division entière)
echo $((a % b))    # 1 (modulo)
```

9. Gestion des erreurs (robustesse)

```
#!/bin/bash
set -e      # arrête le script à la première erreur
set -u      # erreur si variable non définie
set -o pipefail # détecte les erreurs dans les pipes
set -euo pipefail # combinaison recommandée en production

# Vérifier le code de retour
if ! command -v docker &> /dev/null; then
    echo "Docker n'est pas installé !" >&2
    exit 1
fi
```

10. Script complet : exemple DevOps réel

```
#!/bin/bash
set -euo pipefail

# Script de sauvegarde simple
SOURCE="/var/www/monsite"
DEST="/backups"
DATE=$(date +%Y%m%d_%H%M%S)
ARCHIVE="$DEST/backup_$DATE.tar.gz"

echo "[INFO] Démarrage de la sauvegarde..."

# Créer le dossier de destination s'il n'existe pas
mkdir -p "$DEST"

# Créer l'archive compressée
if tar -czf "$ARCHIVE" "$SOURCE"; then
    echo "[OK] Sauvegarde créée : $ARCHIVE"
else
    echo "[ERREUR] Échec de la sauvegarde" >&2
    exit 1
fi

# Supprimer les sauvegardes de plus de 7 jours
find "$DEST" -name "backup_*.tar.gz" -mtime +7 -delete
echo "[INFO] Anciennes sauvegardes nettoyées."

# Taille de la sauvegarde
taille=$(du -h "$ARCHIVE" | cut -f1)
echo "[INFO] Taille : $taille"
```

11. Bonnes pratiques

- Toujours mettre le **shebang** `#!/bin/bash`.
- Utiliser `set -euo pipefail` pour la robustesse.
- Mettre les variables entre **guillemets** : `"$var"`.
- Vérifier les arguments en entrée.
- Écrire les erreurs sur `stderr` avec `>&2`.
- Utiliser `shellcheck` pour valider vos scripts :

```
sudo apt install shellcheck
shellcheck script.sh
```

Exercices

1. Écrivez un script qui prend un nom en argument et affiche « Bonjour ».
2. Écrivez un script qui vérifie si un fichier passé en argument existe.

3. Faites une boucle qui affiche les nombres pairs de 1 à 20.
4. Écrivez une fonction qui teste si un service systemd est actif.
5. Écrivez un script qui compte le nombre d'erreurs « 404 » dans un fichier log.
6. Améliorez le script de sauvegarde pour qu'il envoie un message si l'espace disque est insuffisant.

✓ Passez au [Module 06 — Réseau Fondamentaux](#).

Module 06 — Réseau : Fondamentaux & Modèles

 **Objectif** : comprendre comment les données circulent sur un réseau, base indispensable du DevOps.

1. Qu'est-ce qu'un réseau ?

Un **réseau** est un ensemble d'équipements (ordinateurs, serveurs, routeurs...) interconnectés qui s'échangent des données. Internet est le « réseau des réseaux ».

Types de réseaux : - **LAN** (Local Area Network) : réseau local (maison, bureau) - **WAN** (Wide Area Network) : étendu (entre villes, Internet) - **VLAN** : réseau local virtuel (segmentation logique) - **VPN** : réseau privé virtuel (tunnel sécurisé)

2. Le modèle OSI (7 couches)

Le modèle **OSI** décrit comment les données voyagent, en 7 couches :

#	Couche	Rôle	Exemples
7	Application	Interface utilisateur	HTTP, DNS, SSH, FTP
6	Présentation	Format, chiffrement	TLS/SSL, JPEG
5	Session	Gère les sessions	sockets
4	Transport	Fiabilité, ports	TCP, UDP
3	Réseau	Adressage, routage	IP, ICMP
2	Liaison	Adresses MAC, trames	Ethernet, ARP
1	Physique	Signaux, câbles	câbles, Wi-Fi

 Moyen mnémotechnique : « **All People Seem To Need Data Processing** » (de 7 à 1).

3. Le modèle TCP/IP (4 couches — le vrai)

En pratique, Internet utilise le modèle **TCP/IP**, plus simple :

Couche TCP/IP	Équivaut OSI	Protocoles
Application	5-6-7	HTTP, DNS, SSH, FTP
Transport	4	TCP, UDP
Internet	3	IP, ICMP
Accès réseau	1-2	Ethernet, Wi-Fi

4. L'encapsulation

Quand vous envoyez des données, chaque couche ajoute un **en-tête** (header) :

```
[ Données ]  
[ En-tête TCP | Données ]      → Segment  
[ En-tête IP | TCP | Données ] → Paquet  
[ En-tête Ethernet | IP | TCP | Données ] → Trame
```

À la réception, le processus inverse (décapsulation) retire les en-têtes couche par couche.

5. Adresses MAC vs adresses IP

- **Adresse MAC** : identifiant physique de la carte réseau, gravé en usine (ex : **00:1A:2B:3C:4D:5E**). Unique, ne change pas. Couche 2.
- **Adresse IP** : identifiant logique, attribué par le réseau, peut changer (ex : **192.168.1.10**). Couche 3.

Analogie : la MAC est comme le numéro de série de votre téléphone, l'IP est comme votre numéro de téléphone (qui peut changer d'opérateur).

6. TCP vs UDP (couche transport)

Critère	TCP	UDP
Connexion	Orienté connexion	Sans connexion
Fiabilité	Garantie (accusés de réception)	Aucune garantie
Ordre	Données dans l'ordre	Pas d'ordre garanti
Vitesse	Plus lent	Plus rapide
Usage	Web, mail, SSH, BDD	Streaming, DNS, jeux, VoIP

TCP — Three-way handshake (établissement de connexion) :

```
Client --- SYN --->   Serveur
Client <-- SYN-ACK --- Serveur
Client --- ACK --->   Serveur
      (connexion établie)
```

7. Les ports

Un **port** identifie un service précis sur une machine (0-65535). Une connexion = IP source:port → IP destination:port.

Ports « bien connus » à mémoriser :

Port	Service
20/21	FTP
22	SSH
25	SMTP (mail)
53	DNS
80	HTTP
443	HTTPS
3306	MySQL
5432	PostgreSQL
6379	Redis
8080	HTTP alternatif
27017	MongoDB

💡 Ports < 1024 = privilégiés (nécessitent root pour être ouverts).

8. Les équipements réseau

- **Switch (commutateur)** : relie les machines d'un LAN (couche 2, MAC).
- **Routeur** : relie des réseaux différents, fait du routage (couche 3, IP).
- **Passerelle (gateway)** : porte de sortie vers d'autres réseaux.
- **Load balancer** : répartit le trafic entre plusieurs serveurs.
- **Pare-feu (firewall)** : filtre le trafic selon des règles.
- **Proxy / Reverse proxy** : intermédiaire (Nginx, HAProxy).

9. Le chemin d'une requête web (vue d'ensemble)

Quand vous tapez `https://exemple.com` : 1. **DNS** : `exemple.com` → adresse IP (ex : `93.184.216.34`). 2. **TCP** : ouverture de connexion (handshake) sur le port 443. 3. **TLS** :

négociation du chiffrement (HTTPS). 4. **HTTP** : envoi de la requête `GET /`. 5. Le serveur répond avec le HTML. 6. Le navigateur affiche la page.

🎯 En DevOps, comprendre ce chemin permet de savoir **où** ça casse (DNS ? réseau ? appli ? certificat ?).

Exercices

1. Citez les 7 couches du modèle OSI dans l'ordre.
2. Quelle est la différence entre une adresse MAC et une adresse IP ?
3. Pour télécharger un gros fichier de façon fiable, TCP ou UDP ?
4. Sur quel port tourne HTTPS ? Et SSH ? Et PostgreSQL ?
5. Décrivez les 3 étapes du three-way handshake TCP.
6. Expliquez ce qui se passe quand on tape une URL dans un navigateur.

✅ Passez au [Module 07 — Adressage IP](#).

Module 07 — Réseau : Adressage IP & Sous-réseaux

 **Objectif** : maîtriser l'adressage IPv4, les masques et le subnetting (essentiel pour le cloud et Kubernetes).

1. L'adresse IPv4

Une adresse IPv4 est composée de **4 octets** (32 bits), notés en décimal séparés par des points :

```
192.168.1.10
|   |   |   |
octets (0 à 255 chacun)
```

En binaire :

```
192      168      1      10
11000000 10101000 00000001 00001010
```

Chaque octet = 8 bits $\rightarrow 2^8 = 256$ valeurs (0 à 255).

2. Adresses privées vs publiques

Adresses privées (non routables sur Internet, usage interne) :

Plage	Classe	Usage
10.0.0.0 – 10.255.255.255	A	Grandes entreprises
172.16.0.0 – 172.31.255.255	B	Moyennes
192.168.0.0 – 192.168.255.255	C	Maison / petits réseaux

Adresses publiques : tout le reste, uniques sur Internet, attribuées par les FAI.

Adresses spéciales : - **127.0.0.1** : **localhost** (la machine elle-même, loopback) - **0.0.0.0** : toutes les interfaces / route par défaut - **255.255.255.255** : broadcast

3. Le masque de sous-réseau

Le **masque** sépare l'adresse en deux parties : - **Partie réseau** (network) : identifie le réseau - **Partie hôte** (host) : identifie la machine dans le réseau

```

IP       : 192.168.1.10
Masque   : 255.255.255.0
           \_____/  \____/
            réseau   hôte

```

Le masque **255.255.255.0** signifie : les 3 premiers octets = réseau, le dernier = hôtes.

4. La notation CIDR

CIDR = nombre de bits à 1 dans le masque, noté **/n** :

Masque décimal	CIDR	Nb d'hôtes utilisables
255.0.0.0	/8	16 777 214
255.255.0.0	/16	65 534
255.255.255.0	/24	254
255.255.255.128	/25	126
255.255.255.192	/26	62
255.255.255.224	/27	30
255.255.255.240	/28	14

Exemple : **192.168.1.0/24** = réseau avec 256 adresses (254 utilisables).

 **Formule** : nombre d'hôtes utilisables = $2^{(\text{bits hôte})} - 2$ (on retire l'adresse réseau et l'adresse de broadcast).

5. Adresse réseau, broadcast et plage utilisable

Pour **192.168.1.0/24** : - **Adresse réseau** : **192.168.1.0** (première, non assignable) - **Première utilisable** : **192.168.1.1** - **Dernière utilisable** : **192.168.1.254** - **Broadcast** : **192.168.1.255** (dernière, envoi à tous)

6. Le subnetting (découpage)

Découper un réseau en sous-réseaux plus petits.

Exemple : on a **192.168.1.0/24** et on veut **4 sous-réseaux**. - $4 = 2^2 \rightarrow$ on emprunte **2 bits** à la partie hôte. - Nouveau masque : **/24 + 2 = /26** $\rightarrow$ **255.255.255.192** - Chaque sous-réseau a $2^{(8-2)} = 64$ adresses (62 utilisables).

Sous-réseau	Plage	Broadcast
192.168.1.0/26	.1 - .62	.63
192.168.1.64/26	.65 - .126	.127
192.168.1.128/26	.129 - .190	.191
192.168.1.192/26	.193 - .254	.255

🎯 **Pourquoi c'est utile en DevOps ?** Dans AWS/Azure/GCP, vous créez des **VPC** et des **subnets** avec ces notations CIDR. Kubernetes attribue aussi des plages CIDR aux pods et services.

7. La passerelle par défaut (gateway)

C'est l'adresse du routeur qui permet de sortir du réseau local. Souvent **.1** ou **.254**.

```
Machine : 192.168.1.10/24
Gateway : 192.168.1.1
```

Tout trafic vers une IP hors du réseau local passe par la gateway.

8. IPv6 (notion)

IPv4 est limité (~4 milliards d'adresses, épuisées). IPv6 utilise **128 bits** :

```
2001:0db8:85a3:0000:0000:8a2e:0370:7334
```

- Notation hexadécimale, 8 groupes de 4
- Les zéros peuvent être compressés : **2001:db8:85a3::8a2e:370:7334**
- **::1** = localhost en IPv6

9. NAT (Network Address Translation)

Le **NAT** permet à plusieurs machines d'un réseau privé de partager une seule IP publique. Votre box fait du NAT : toutes vos machines (192.168.x.x) sortent sur Internet avec une seule IP publique.

🎯 En cloud, le NAT Gateway permet aux instances privées d'accéder à Internet sans être exposées.

10. Configurer une IP sous Linux

```
ip addr show          # afficher les adresses IP
ip a                  # raccourci
ip route show         # table de routage
ip route get 8.8.8.8   # quelle route pour cette IP ?

# Configuration temporaire
sudo ip addr add 192.168.1.50/24 dev eth0
sudo ip route add default via 192.168.1.1

# Configuration permanente (Ubuntu = netplan)
# /etc/netplan/00-config.yaml
```

Exemple netplan :

```
network:
  version: 2
  ethernets:
    eth0:
      addresses: [192.168.1.50/24]
      routes:
        - to: default
          via: 192.168.1.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

Exercices

1. Pour le réseau `10.0.0.0/8`, combien d'hôtes utilisables ?
2. Donnez l'adresse réseau et le broadcast de `192.168.10.0/24`.
3. Découpez `192.168.0.0/24` en 2 sous-réseaux. Donnez leurs plages.
4. Quelle est l'adresse de loopback ? À quoi sert-elle ?
5. `172.20.5.3` est-elle une adresse privée ou publique ?
6. Quel masque décimal correspond à `/27` ? Combien d'hôtes utilisables ?

✓ Passez au [Module 08 — Protocoles & Services](#).

Module 07b — L'adressage IP expliqué pas à pas (+ exercices corrigés)

🎯 **Objectif** : comprendre l'adressage IPv4 **en profondeur** et savoir calculer n'importe quel sous-réseau à la main. Ce module complète le [Module 07](#).

PARTIE 1 — Comprendre ce qu'est une adresse IP

1.1 L'idée de base

Une adresse IP, c'est comme une **adresse postale** pour une machine. Elle doit être **unique** dans son réseau pour que les données arrivent au bon endroit.

Une adresse IPv4 = **32 bits**, découpés en **4 octets** (4 paquets de 8 bits) :

```
192      .    168      .      1      .    10
11000000 . 10101000 . 00000001 . 00001010
octet 1   octet 2   octet 3   octet 4
```

Chaque octet va de **0 à 255** (car 8 bits $\rightarrow 2^8 = 256$ valeurs possibles).

1.2 Convertir binaire $\leftrightarrow$ décimal (la base de tout)

Chaque bit d'un octet a un **poids** :

```
Poids : 128 64 32 16 8 4 2 1
Bit   : 1  1  0  0  0  0  0  0 = 128 + 64 = 192
```

Méthode décimal $\rightarrow$ binaire : on soustrait du plus grand poids au plus petit.

Exemple : convertir **168**

```
168 ≥ 128 ? oui → bit=1, reste 168-128 = 40
40  ≥ 64  ? non → bit=0
40  ≥ 32  ? oui → bit=1, reste 40-32 = 8
8   ≥ 16  ? non → bit=0
8   ≥ 8   ? oui → bit=1, reste 0
0   ≥ 4   ? non → bit=0
0   ≥ 2   ? non → bit=0
0   ≥ 1   ? non → bit=0
Résultat : 10101000
```

Méthode binaire $\rightarrow$ décimal : on additionne les poids des bits à 1.

10101000 = 128 + 32 + 8 = 168 ✓

💡 Apprenez par cœur les poids : **128 - 64 - 32 - 16 - 8 - 4 - 2 - 1**. C'est l'outil n°1 du subnetting.

PARTIE 2 — Le masque de sous-réseau

2.1 À quoi sert le masque ?

Une adresse IP contient **2 informations** mélangées : - la **partie réseau** : « dans quel quartier j'habite » - la **partie hôte** : « quel numéro de maison »

Le **masque** indique où s'arrête le réseau et où commence l'hôte. Les bits à **1** du masque = partie **réseau**. Les bits à **0** = partie **hôte**.

```
IP      : 192.168.1.10  = 11000000.10101000.00000001.00001010
Masque  : 255.255.255.0 = 11111111.11111111.11111111.00000000
                        \_____ réseau _____/\_ hôte _/
```

2.2 La notation CIDR (/n)

Au lieu d'écrire le masque en entier, on écrit le **nombre de bits à 1** :

```
255.255.255.0 → /24  (24 bits à 1)
255.255.0.0   → /16
255.0.0.0     → /8
255.255.255.192 → /26
```

Donc **192.168.1.10/24** veut dire : IP **192.168.1.10** avec un masque de 24 bits.

2.3 Tableau des masques (à garder sous la main)

CIDR	Masque décimal	Bits hôte	Hôtes utilisables
/24	255.255.255.0	8	254
/25	255.255.255.128	7	126
/26	255.255.255.192	6	62
/27	255.255.255.224	5	30
/28	255.255.255.240	4	14
/29	255.255.255.248	3	6
/30	255.255.255.252	2	2

📌 Les valeurs du dernier octet des masques : **128, 192, 224, 240, 248, 252, 254, 255**. Mémorisez-les, elles reviennent tout le temps.

PARTIE 3 — Les 4 nombres clés d'un réseau

Pour **tout** réseau, on calcule 4 choses :

1. **Adresse réseau** : la première adresse → identifie le réseau (non assignable).
2. **Première adresse utilisable** : adresse réseau + 1.
3. **Dernière adresse utilisable** : broadcast – 1.
4. **Adresse de broadcast** : la dernière adresse → envoie à tout le monde (non assignable).

Formule du nombre d'hôtes

Nombre d'hôtes utilisables = $2^{(\text{bits hôte})} - 2$

Le **-2** correspond à l'adresse réseau et au broadcast, qu'on ne peut pas donner à une machine.

Exemple **/24** : bits hôte = 8 → $2^8 - 2 = 256 - 2 = \mathbf{254 \text{ hôtes}}$.

PARTIE 4 — Méthode universelle de calcul (à appliquer toujours)

Voici **LA** méthode à suivre pour n'importe quel exercice de subnetting.

Donnée : une IP + un CIDR, par exemple **192.168.1.100/26**.

Étape 1 — Trouver le masque et l'octet « intéressant ». **/26** → masque **255.255.255.192**. Le 4^e octet (192) est l'octet qui change.

Étape 2 — Calculer le « pas » (block size).

Pas = 256 – valeur de l'octet du masque
Pas = 256 – 192 = 64

Les réseaux vont donc de 64 en 64 : **0, 64, 128, 192**.

Étape 3 — Situer l'IP dans le bon bloc. L'IP finit par `.100`. Entre quels multiples de 64 ? → entre 64 et 128. Donc **adresse réseau = .64**.

Étape 4 — En déduire les 4 nombres.

```
Adresse réseau      : 192.168.1.64
1re utilisable     : 192.168.1.65
Dernière utilisable : 192.168.1.126
Broadcast          : 192.168.1.127 (réseau suivant .128, donc broadcast = .127)
```

✓ Avec ces 4 étapes (masque → pas → bloc → 4 nombres), vous résolvez **tout**.

PARTIE 5 — Le subnetting : découper un réseau

5.1 Pourquoi découper ?

- **Sécurité** : isoler des groupes (serveurs, postes, invités).
- **Performance** : réduire les domaines de broadcast.
- **Organisation** : un sous-réseau par service / par zone cloud.

5.2 Découper selon un nombre de SOUS-RÉSEAUX

On **emprunte des bits** à la partie hôte. Pour obtenir N sous-réseaux, il faut **b** bits tels que $2^b \geq N$.

Sous-réseaux voulus	Bits à emprunter
2	1
4	2
8	3
16	4
32	5

Exemple : découper `192.168.1.0/24` en **4** sous-réseaux. - $4 = 2^2 \rightarrow$ emprunter **2 bits** → nouveau masque `/24 + 2 = /26`. - Pas = $256 - 192 = 64$. - Sous-réseaux : `.0`, `.64`, `.128`, `.192`.

5.3 Découper selon un nombre d'HÔTES

On garde **assez de bits hôte** pour le besoin : $2^h - 2 \geq \text{hôtes voulus}$.

Exemple : il me faut un réseau pour **50 machines**. - $2^5 - 2 = 30$ ✗ (trop peu) → $2^6 - 2 = 62$ ✓ - 6 bits hôte → masque `/26`.

PARTIE 6 — EXERCICES CORRIGÉS

Essayez **avant** de dérouler la solution. Prenez un papier et appliquez la méthode des 4 étapes.

● Niveau 1 — Lecture d'adresse

Exercice 1.1 — `10.20.30.40` est-elle privée ou publique ?

Solution

`10.x.x.x` appartient à la plage privée `10.0.0.0 – 10.255.255.255` → **privée**.

Exercice 1.2 — Convertir `192` et `240` en binaire.

Solution

- `192` = $128 + 64$ = `11000000`
- `240` = $128 + 64 + 32 + 16$ = `11110000`

Exercice 1.3 — Convertir `10101100` en décimal.

Solution

$128 + 32 + 8 + 4$ = **172**.

● Niveau 2 — Réseau / Broadcast

Exercice 2.1 — Pour `192.168.10.0/24`, donnez adresse réseau, 1^{re} et dernière utilisable, broadcast.

Solution

- Masque `/24` → pas = $256 - 255$... non : le 4^e octet du masque vaut 0, on prend tout l'octet.
- Réseau : `192.168.10.0`
- 1^{re} utilisable : `192.168.10.1`
- Dernière utilisable : `192.168.10.254`
- Broadcast : `192.168.10.255`
- Hôtes : $2^8 - 2$ = **254**

Exercice 2.2 — Combien d'hôtes utilisables dans un `/16` ?

Solution

Bits hôte = $32 - 16 = 16 \rightarrow 2^{16} - 2 = 65\,536 - 2 = \mathbf{65\,534}$.

Exercice 2.3 — Quel est le masque décimal d'un `/27` ? Combien d'hôtes ?

Solution

- `/27` → 3 octets pleins + 3 bits → 4^e octet = 11100000 = 224 → `255.255.255.224`
- Bits hôte = 5 → $2^5 - 2 = \mathbf{30}$ hôtes

● Niveau 3 — Méthode des 4 étapes

Exercice 3.1 — Pour l'IP `192.168.1.100/26`, donnez les 4 nombres du réseau auquel elle appartient.

Solution

1. Masque `/26` = `255.255.255.192`, octet intéressant = 4^e.
2. Pas = $256 - 192 = \mathbf{64}$ → blocs : 0, 64, 128, 192.
3. `.100` est entre 64 et 128 → réseau = **.64**. 4. - Réseau : `192.168.1.64` - 1^{re} utilisable : `192.168.1.65` - Dernière utilisable : `192.168.1.126` - Broadcast : `192.168.1.127`

Exercice 3.2 — Pour `172.16.5.200/28`, donnez réseau et broadcast.

Solution

1. `/28` = `255.255.255.240`, pas = $256 - 240 = \mathbf{16}$.
2. Blocs : 0, 16, 32, ... 192, **208**, ... `.200` est entre 192 et 208 → réseau = **.192**. 3. - Réseau : `172.16.5.192` - Broadcast : `172.16.5.207` (réseau suivant .208 → broadcast = .207) - Utilisables : `.193` à `.206` (14 hôtes)

Exercice 3.3 — `10.0.0.130/25` appartient à quel réseau ?

Solution

1. `/25` = `255.255.255.128`, pas = $256 - 128 = \mathbf{128}$.
2. Blocs : 0, 128. `.130` est dans le bloc **.128**. 3. - Réseau : `10.0.0.128` - 1^{re} utilisable : `10.0.0.129` - Dernière : `10.0.0.254` - Broadcast : `10.0.0.255`

● Niveau 4 — Subnetting (découpage)

Exercice 4.1 — Découpez `192.168.0.0/24` en **4 sous-réseaux égaux**. Donnez-les tous.

Solution

- $4 = 2^2 \rightarrow$ emprunter 2 bits $\rightarrow$ /26, pas = 64.

#	Réseau	Plage utilisable	Broadcast
1	192.168.0.0/26	.1 - .62	.63
2	192.168.0.64/26	.65 - .126	.127
3	192.168.0.128/26	.129 - .190	.191
4	192.168.0.192/26	.193 - .254	.255

Exercice 4.2 — Vous avez 172.16.0.0/16 et devez créer **8 sous-réseaux**. Masque ? 3 premiers sous-réseaux ?

Solution

- $8 = 2^3 \rightarrow$ emprunter 3 bits $\rightarrow$ /16 + 3 = /19 $\rightarrow$ masque 255.255.224.0.
- L'octet intéressant est le 3^e, pas = $256 - 224 = 32$.

#	Réseau
1	172.16.0.0/19
2	172.16.32.0/19
3	172.16.64.0/19

Chaque sous-réseau : $2^{(32-19)} - 2 = 2^{13} - 2 = \mathbf{8190}$ hôtes.

Exercice 4.3 — Il vous faut un réseau pour **exactement 100 machines**. Quel est le plus petit masque possible ?

Solution

- $2^6 - 2 = 62$ ❌ ; $2^7 - 2 = 126$ ✅ $\rightarrow$ besoin de **7 bits hôte**.
- Masque = $32 - 7 =$ /25 (255.255.255.128).
- Capacité : 126 hôtes (couvre les 100).

Exercice 4.4 (synthèse) — Une entreprise reçoit 192.168.50.0/24. Elle veut : - un sous-réseau pour 60 PC, - un sous-réseau pour 25 imprimantes, - un sous-réseau pour 10 serveurs.

Proposez un découpage (VLSM — masques de tailles variables).

Solution

On commence par le plus gros besoin (VLSM = on alloue du plus grand au plus petit).

1) 60 PC → $2^6 - 2 = 62$ ✓ → /26 (pas 64) - 192.168.50.0/26 → .1 à .62 (broadcast .63)

2) 25 imprimantes → $2^5 - 2 = 30$ ✓ → /27 (pas 32) - On continue après .63 → 192.168.50.64/27 → .65 à .94 (broadcast .95)

3) 10 serveurs → $2^4 - 2 = 14$ ✓ → /28 (pas 16) - On continue après .95 → 192.168.50.96/28 → .97 à .110 (broadcast .111)

Usage	Réseau	Masque	Plage	Broadcast
PC (60)	192.168.50.0	/26	.1-.62	.63
Imprimantes (25)	192.168.50.64	/27	.65-.94	.95
Serveurs (10)	192.168.50.96	/28	.97-.110	.111

Il reste de l'espace libre à partir de 192.168.50.112 pour de futurs réseaux. 🎉

PARTIE 7 — Vérifier ses calculs sous Linux

Inutile de tout faire à la main en production : des outils valident vos calculs.

```
# ipcalc : l'outil idéal d'apprentissage
sudo apt install ipcalc
ipcalc 192.168.1.100/26
# Affiche : Network, HostMin, HostMax, Broadcast, nb d'hôtes...

ipcalc 192.168.0.0/24 -s 60 25 10 # propose un découpage VLSM

# Voir sa propre config
ip a
ip route
```

Exemple de sortie `ipcalc 192.168.1.100/26` :

```
Address: 192.168.1.100
Netmask: 255.255.255.192 = 26
Network: 192.168.1.64/26
HostMin: 192.168.1.65
HostMax: 192.168.1.126
Broadcast: 192.168.1.127
Hosts/Net: 62
```

🎯 **En DevOps**, ces calculs servent directement à dimensionner un **VPC AWS**, un **subnet Azure**, ou la plage **CIDR des pods** dans Kubernetes.

PARTIE 8 — Mémo express (à imprimer)

```
Poids des bits      : 128 64 32 16 8 4 2 1
Valeurs de masque   : 128 192 224 240 248 252 254 255
Hôtes utilisables    : 2^(bits hôte) - 2
Pas (block size)     : 256 - octet du masque
```

Méthode 4 étapes :

1. Masque + octet intéressant
2. Pas = 256 - octet du masque
3. Trouver le bloc qui contient l'IP → adresse réseau
4. Réseau / 1^{re} / dernière / broadcast

CIDR	Masque	Hôtes
/24	.0	254
/25	.128	126
/26	.192	62
/27	.224	30
/28	.240	14
/29	.248	6
/30	.252	2

Exercices supplémentaires (sans solution — entraînez-vous !)

1. `10.10.10.77/26` → réseau, broadcast, plage utilisable.
2. Découpez `192.168.100.0/24` en 8 sous-réseaux. Donnez les 8.
3. Il vous faut 500 hôtes : quel masque minimum ?
4. `172.16.20.130/25` appartient à quel réseau ?
5. Convertissez `255.255.255.248` en notation CIDR.
6. VLSM sur `10.0.0.0/24` pour : 100 hôtes, 50 hôtes, 20 hôtes, 2 hôtes (liaison).

💡 Vérifiez vos réponses avec `ipcalc`.

⬅️ Retour au [Module 07](#) ou au [sommaire](#).

Module 08 — Réseau : Protocoles & Services

 **Objectif** : comprendre les protocoles applicatifs que tout DevOps manipule au quotidien.

1. DNS (Domain Name System)

Le DNS traduit les **noms de domaine** en **adresses IP**. `exemple.com` → `93.184.216.34`

Hiérarchie DNS :

```
graph TD
    Root["."] --- com["com"]
    com --- exemple["exemple"]
    exemple --- www["www"]
    Root --- Racine["(racine)"]
    com --- TLD["(TLD - Top Level Domain)"]
    exemple --- domaine["(domaine)"]
    www --- sous-domaine["(sous-domaine)"]
```

Types d'enregistrements DNS :

Type	Rôle
A	Nom → IPv4
AAAA	Nom → IPv6
CNAME	Alias vers un autre nom
MX	Serveur de mail
TXT	Texte (SPF, vérifications)
NS	Serveur de noms
PTR	IP → Nom (reverse)

Commandes DNS :

```
nslookup exemple.com      # résolution simple
dig exemple.com            # détaillé
dig exemple.com +short     # juste l'IP
dig MX gmail.com           # serveurs de mail
host exemple.com           # alternative
cat /etc/resolv.conf       # serveurs DNS configurés
cat /etc/hosts             # résolution locale statique
```

 `/etc/hosts` permet de forcer une résolution locale (très utile en dev/test).

2. DHCP (Dynamic Host Configuration Protocol)

Le DHCP attribue **automatiquement** une configuration IP aux machines : IP, masque, passerelle, DNS.

Processus DORA :

```
Client → DISCOVER (qui peut me donner une IP ?)
Serveur → OFFER (voici une IP proposée)
Client → REQUEST (je prends celle-là)
Serveur → ACK (validé)
```

3. HTTP / HTTPS

HTTP (HyperText Transfer Protocol) est le protocole du web (port 80). **HTTPS** = HTTP + chiffrement TLS (port 443).

Méthodes HTTP :

Méthode	Usage
GET	Lire une ressource
POST	Créer une ressource
PUT	Remplacer une ressource
PATCH	Modifier partiellement
DELETE	Supprimer
HEAD	En-têtes seulement

Codes de statut HTTP (à connaître par cœur) :

Code	Signification
200	OK
201	Created
301/302	Redirection
400	Bad Request
401	Non authentifié
403	Interdit
404	Non trouvé
500	Erreur serveur
502	Bad Gateway (proxy)
503	Service indisponible

🎯 En DevOps, un **502/503** indique souvent un backend en panne derrière un reverse proxy.

Tester avec curl :

```
curl https://exemple.com           # récupère le contenu
curl -I https://exemple.com        # en-têtes seulement
curl -v https://exemple.com        # mode verbeux (debug)
curl -X POST -d '{"k":"v"}' \
  -H "Content-Type: application/json" \
  https://api.exemple.com/data     # requête POST
curl -o page.html https://exemple.com # sauver dans un fichier
curl -w "%{http_code}" -s -o /dev/null https://exemple.com # juste le code
```

4. SSH (Secure Shell)

SSH permet de se connecter à distance de façon **sécurisée** (port 22). C'est l'outil n°1 du DevOps.

```
ssh utilisateur@serveur           # connexion
ssh -p 2222 user@serveur          # port personnalisé
ssh -i ~/.ssh/cle.pem user@serveur # avec une clé
```

Authentification par clé (recommandée) :

```
ssh-keygen -t ed25519 -C "mon-email" # générer une paire de clés
ssh-copy-id user@serveur              # copier la clé publique
# puis connexion sans mot de passe
```

- Clé **privée** (`id_ed25519`) : reste secrète sur votre machine.
- Clé **publique** (`id_ed25519.pub`) : copiée sur le serveur.

Copier des fichiers via SSH :

```
scp fichier.txt user@serveur:/chemin/ # envoyer un fichier
scp user@serveur:/chemin/fichier.txt . # récupérer
rsync -avz dossier/ user@serveur:/dest/ # synchronisation efficace
```

Tunnel SSH :

```
ssh -L 8080:localhost:80 user@serveur # port local → distant
```

5. TLS/SSL et les certificats

TLS chiffre les communications (HTTPS, etc.). Repose sur un système de certificats.

- Un **certificat** prouve l'identité d'un serveur.

- Émis par une **autorité de certification (CA)**.
- **Let's Encrypt** fournit des certificats gratuits (via `certbot`).

```
# Vérifier un certificat
openssl s_client -connect exemple.com:443
echo | openssl s_client -connect exemple.com:443 2>/dev/null | openssl x509 -noout -dates
```

🔗 En DevOps : gérer le renouvellement automatique des certificats est crucial (un certificat expiré = site down).

6. FTP / SFTP

- **FTP** (port 21) : transfert de fichiers **non chiffré** (à éviter).
- **SFTP** : transfert sécurisé via SSH (recommandé).

```
sftp user@serveur
> put fichier.txt
> get fichier.txt
> ls
```

7. Les serveurs web & reverse proxy

Nginx et **Apache** servent des sites web et font office de reverse proxy.

Configuration Nginx minimale (`/etc/nginx/sites-available/monsite`) :

```
server {
    listen 80;
    server_name exemple.com;

    location / {
        proxy_pass http://localhost:3000;    # vers l'app backend
        proxy_set_header Host $host;
    }
}
```

```
sudo nginx -t          # tester la config
sudo systemctl reload nginx    # appliquer
```

8. Les API REST

La plupart des services modernes communiquent via des **API REST** (HTTP + JSON).

```
# GET
curl https://api.github.com/users/torvalds

# POST avec authentification
curl -X POST https://api.exemple.com/login \
  -H "Authorization: Bearer TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"user": "admin"}'
```

💡 `jq` permet de parser le JSON :

```
curl -s https://api.github.com/users/torvalds | jq '.name'
```

Exercices

1. Résolvez l'adresse IP de `github.com` avec `dig +short`.
2. Quels sont les serveurs mail (MX) de `gmail.com` ?
3. Récupérez uniquement le code HTTP retourné par un site avec `curl`.
4. Générez une paire de clés SSH ed25519.
5. À quoi correspond un code HTTP 502 ? Et 401 ?
6. Écrivez une commande curl qui fait un POST JSON vers une API.

✅ Passez au [Module 09 — Outils de diagnostic](#).

Module 09 — Réseau : Diagnostic & Outils Linux

 **Objectif** : savoir diagnostiquer un problème réseau, compétence quotidienne du DevOps.

1. La méthode de diagnostic (couche par couche)

Quand « ça ne marche pas », remontez les couches : 1. **Physique/interface** : la carte réseau est-elle UP ? (`ip a`) 2. **IP** : ai-je une adresse, une route ? (`ip a` , `ip route`) 3. **Connectivité** : puis-je joindre la machine ? (`ping`) 4. **DNS** : le nom se résout-il ? (`dig` , `nslookup`) 5. **Port/Service** : le port est-il ouvert ? (`ss` , `nc` , `telnet`) 6. **Application** : l'appli répond-elle ? (`curl`)

2. Voir la configuration réseau

```
ip a                # adresses IP des interfaces
ip link             # état des interfaces (UP/DOWN)
ip route             # table de routage
ip neigh            # table ARP (voisins)
hostname -I          # IP de la machine
cat /etc/resolv.conf # serveurs DNS

# Anciennes commandes (paquet net-tools, encore courantes)
ifconfig             # config interfaces
route -n             # routage
netstat -tulpn       # ports en écoute
```

3. Tester la connectivité : ping

`ping` envoie des paquets ICMP pour tester si une machine répond.

```
ping google.com      # test continu (Ctrl+C pour arrêter)
ping -c 4 google.com  # 4 paquets seulement
ping -c 4 8.8.8.8     # tester par IP (isole le DNS)
```

 Si `ping 8.8.8.8` marche mais `ping google.com` échoue → **problème DNS**.

4. Tracer le chemin : traceroute

Affiche tous les routeurs (sauts) jusqu'à la destination.

```
tracert google.com      # chemin complet
tracert -n google.com   # variante sans root
mtr google.com          # traceroute + ping en continu (top !)
```

5. Tester un port : la vraie question DevOps

« Le port est-il ouvert et joignable ? »

```
# Avec netcat (nc)
nc -zv serveur 22      # teste le port 22
nc -zv 192.168.1.10 80  # teste le port 80

# Avec telnet
telnet serveur 443

# Test bash pur (sans outil)
timeout 2 bash -c "echo > /dev/tcp/serveur/22" && echo "ouvert" || echo "fermé"

# Scanner plusieurs ports avec nmap
nmap -p 1-1000 serveur  # scan de ports
nmap -sV serveur        # détecte les versions de services
```

6. Voir les connexions et ports en écoute : ss

ss (remplace **netstat**) montre les sockets/connexions.

```
ss -tulpn      # TCP+UDP en écoute, avec processus et ports
ss -t          # connexions TCP établies
ss -tan        # toutes les connexions TCP (numérique)
ss -s          # statistiques résumées
ss -tp state established # connexions actives
```

Options : **-t** TCP, **-u** UDP, **-l** listening, **-p** processus, **-n** numérique.

 **Cas typique** : « mon app écoute-t-elle bien sur le port 3000 ? »

```
ss -tulpn | grep 3000
```

7. Analyser le trafic : tcpdump

Capture les paquets réseau pour un diagnostic approfondi.

```
sudo tcpdump -i eth0          # capture sur eth0
sudo tcpdump -i any port 80    # uniquement le port 80
sudo tcpdump -i any host 192.168.1.10 # une IP précise
sudo tcpdump -i any port 443 -w capture.pcap # sauver pour Wireshark
sudo tcpdump -i any -n port 53 # voir les requêtes DNS
```

💡 Le fichier `.pcap` s'analyse ensuite dans **Wireshark** (interface graphique).

8. Télécharger / tester des URL : curl & wget

```
# curl (le couteau suisse)
curl -I https://exemple.com          # en-têtes
curl -v https://exemple.com          # debug complet
curl -L https://exemple.com          # suit les redirections
curl --resolve exemple.com:443:1.2.3.4 https://exemple.com # forcer une IP
curl -w "DNS:%{time_namelookup} Connect:%{time_connect} Total:%{time_total}\n" \
-s -o /dev/null https://exemple.com # mesurer les temps

# wget
wget https://exemple.com/fichier.zip # télécharger
wget -c https://...                  # reprendre un téléchargement
```

9. Le pare-feu local en diagnostic

Parfois « le port est fermé » = le pare-feu bloque.

```
sudo iptables -L -n -v    # règles iptables
sudo ufw status verbose   # statut UFW (Ubuntu)
sudo firewall-cmd --list-all # firewalld (RHEL)
```

(Détailé dans le [Module 10](#).)

10. Scénarios de diagnostic concrets

Scénario A — « Le site ne répond pas »

```
ping -c2 site.com          # 1. réseau OK ?
dig +short site.com         # 2. DNS résout ?
nc -zv site.com 443         # 3. port ouvert ?
curl -I https://site.com    # 4. appli répond ?
```

Scénario B — « Mon conteneur ne joint pas la BDD »

```
ss -tulpn | grep 5432      # la BDD écoute-t-elle ?
nc -zv db-host 5432        # joignable depuis ici ?
ping db-host               # résolution + réseau ?
```

Scénario C — « Lenteur réseau »

```
mtr destination          # où est la latence ?  
curl -w "@format.txt" ... # décomposer les temps
```

11. Tableau récapitulatif des outils

Besoin	Outil
Voir mon IP / routes	<code>ip a</code> , <code>ip route</code>
Tester si une machine répond	<code>ping</code>
Tracer le chemin	<code>traceroute</code> , <code>mtr</code>
Tester un port	<code>nc -zv</code> , <code>nmap</code>
Voir mes ports ouverts	<code>ss -tulpn</code>
Résoudre un nom	<code>dig</code> , <code>nslookup</code>
Tester une URL HTTP	<code>curl</code> , <code>wget</code>
Capturer le trafic	<code>tcpdump</code> , Wireshark

Exercices

1. Affichez l'adresse IP et la passerelle par défaut de votre machine.
2. Vérifiez si `8.8.8.8` répond avec exactement 3 paquets.
3. Listez tous les ports en écoute sur votre machine avec le nom des processus.
4. Testez si le port 443 de `github.com` est ouvert avec `nc`.
5. Tracez le chemin réseau jusqu'à `1.1.1.1`.
6. Avec `curl`, mesurez le temps total de chargement d'un site web.

✓ Passez au [Module 10 — Sécurité & Pare-feu](#).

Module 10 — Sécurité & Pare-feu

 **Objectif** : sécuriser un serveur Linux et filtrer le trafic réseau.

1. Principes de sécurité (defense in depth)

- **Moindre privilège** : chaque user/service n'a que les droits strictement nécessaires.
- **Surface d'attaque minimale** : fermer tout ce qui n'est pas utilisé.
- **Défense en profondeur** : plusieurs couches (pare-feu + auth + chiffrement).
- **Mises à jour** : appliquer régulièrement les correctifs de sécurité.
- **Journalisation** : tout tracer pour détecter et investiguer.

2. Le pare-feu : concept

Un pare-feu filtre le trafic selon des règles : **autoriser** ou **bloquer** en fonction de l'IP source/destination, du port, du protocole.

Politique recommandée : **deny by default** (tout bloquer, puis autoriser uniquement le nécessaire).

3. UFW (Uncomplicated Firewall — Ubuntu/Debian)

Le plus simple pour débuter.

```
sudo ufw status          # état
sudo ufw enable          # activer
sudo ufw disable         # désactiver

# Politique par défaut
sudo ufw default deny incoming # bloquer tout en entrée
sudo ufw default allow outgoing # autoriser tout en sortie

# Autoriser des services
sudo ufw allow 22/tcp      # SSH
sudo ufw allow 80/tcp      # HTTP
sudo ufw allow 443/tcp     # HTTPS
sudo ufw allow from 192.168.1.0/24 # depuis un réseau
sudo ufw allow from 1.2.3.4 to any port 22 # SSH depuis une IP précise

# Supprimer une règle
sudo ufw delete allow 80/tcp
sudo ufw status numbered  # voir avec numéros
```

 Avant d'activer UFW à distance, **autorisez SSH** sinon vous vous bloquez !

4. firewalld (RHEL / CentOS / Rocky)

```
sudo firewall-cmd --state
sudo firewall-cmd --list-all
sudo firewall-cmd --add-service=http --permanent
sudo firewall-cmd --add-port=8080/tcp --permanent
sudo firewall-cmd --reload
sudo firewall-cmd --remove-port=8080/tcp --permanent
```

`--permanent` rend la règle persistante après redémarrage.

5. iptables (bas niveau, universel)

iptables est le moteur sous-jacent. Plus complexe mais incontournable.

```
sudo iptables -L -n -v          # lister les règles
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT # autoriser SSH
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT # autoriser HTTP
sudo iptables -A INPUT -i lo -j ACCEPT             # loopback
sudo iptables -P INPUT DROP                        # politique par défaut : bloquer
sudo iptables -D INPUT 1                          # supprimer la règle n°1
```

Concepts : **chaînes** (INPUT, OUTPUT, FORWARD), **cibles** (ACCEPT, DROP, REJECT).

 `nftables` est le successeur moderne d'iptables.

6. Sécuriser SSH (priorité absolue)

SSH est la porte d'entrée des serveurs → cible n°1 des attaques.

Fichier `/etc/ssh/sshd_config` :

```
PermitRootLogin no          # interdire la connexion root directe
PasswordAuthentication no   # forcer l'authentification par clé
PubkeyAuthentication yes
Port 2222                   # changer le port (réduit le bruit)
AllowUsers deploy           # limiter aux users autorisés
MaxAuthTries 3              # limiter les tentatives
```

```
sudo systemctl restart sshd # appliquer
```

Bonnes pratiques SSH : - Toujours utiliser des **clés** (jamais de mot de passe). - Désactiver le login root. - Installer **fail2ban** pour bannir les IP qui brute-forcent.

```
sudo apt install fail2ban
sudo systemctl enable --now fail2ban
sudo fail2ban-client status sshd
```

7. Gestion des secrets

⚠ **Ne jamais** mettre de mots de passe / clés API en clair dans le code ou Git.

Bonnes pratiques : - Variables d'environnement (`.env` non commité, dans `.gitignore`). - Gestionnaires de secrets : **HashiCorp Vault**, **AWS Secrets Manager**, **Sealed Secrets** (Kubernetes). - Permissions strictes sur les fichiers de secrets (`chmod 600`).

```
# Vérifier qu'aucun secret n'est commité
git log -p | grep -i "password"
```

8. Mises à jour de sécurité

```
sudo apt update && sudo apt upgrade -y      # Debian/Ubuntu
sudo unattended-upgrades                    # mises à jour auto
sudo dnf update --security                   # RHEL : sécurité seule
```

9. Durcissement de base d'un serveur (checklist)

- [] Mettre à jour le système
- [] Créer un user non-root avec sudo
- [] Désactiver le login root SSH
- [] Authentification SSH par clé uniquement
- [] Activer le pare-feu (deny par défaut)
- [] Installer fail2ban
- [] Fermer les ports/services inutiles
- [] Activer les mises à jour automatiques de sécurité
- [] Configurer la journalisation
- [] Sauvegardes régulières testées

10. Surveiller la sécurité

```
last                # dernières connexions
lastb               # tentatives échouées
who                 # connectés actuellement
sudo grep "Failed password" /var/log/auth.log # échecs SSH
sudo ss -tulpn      # ports ouverts (réduire la surface)
sudo journalctl -u sshd | grep -i fail
```

Exercices

1. Configurez UFW : bloquer tout en entrée, autoriser SSH, HTTP et HTTPS.
2. Listez vos règles UFW avec leurs numéros.
3. Dans `sshd_config`, quelles 2 options désactiveriez-vous en priorité ?
4. Installez fail2ban et vérifiez le statut de la jail SSH.
5. Trouvez les tentatives de connexion SSH échouées dans les logs.
6. Listez tous les ports ouverts et identifiez ceux que vous pourriez fermer.

✓ Passez au [Module 11 — Mise en pratique DevOps](#).

Module 11 — Mise en pratique DevOps

 **Objectif** : relier Linux + Réseau dans des cas concrets d'infrastructure DevOps.

1. Docker : Linux + réseau en action

Docker isole des applications dans des **conteneurs** (basés sur les namespaces et cgroups du noyau Linux).

```
# Installation rapide (Ubuntu)
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER # éviter sudo (reconnexion requise)

# Commandes de base
docker ps                # conteneurs en cours
docker ps -a             # tous les conteneurs
docker images            # images locales
docker run -d -p 8080:80 nginx # lancer nginx, port 8080 → 80
docker logs <id>         # voir les logs
docker exec -it <id> bash  # entrer dans un conteneur
docker stop <id> / docker rm <id>
docker build -t monapp .  # construire une image
```

Le réseau dans Docker :

```
docker network ls        # réseaux Docker
docker network create mon-reseau # créer un réseau
docker run --network mon-reseau ... # connecter un conteneur
```

 **-p 8080:80** = **port mapping** : relie le port 8080 de l'hôte au port 80 du conteneur. C'est ici que vos connaissances réseau (ports, NAT) servent directement.

2. Dockerfile : automatiser une image

```
FROM node:20-alpine      # image de base légère (Alpine Linux)
WORKDIR /app              # dossier de travail
COPY package*.json ./    # copier les dépendances
RUN npm install           # installer
COPY . .                  # copier le code
EXPOSE 3000               # documenter le port
CMD ["node", "server.js"] # commande de démarrage
```

```
docker build -t monapp:1.0 .
docker run -d -p 3000:3000 monapp:1.0
```

3. docker-compose : orchestrer plusieurs services

`docker-compose.yml` — exemple app web + base de données :

```
version: "3.8"
services:
  web:
    build: .
    ports:
      - "3000:3000"
    environment:
      - DB_HOST=db
    depends_on:
      - db
    networks:
      - app-net

  db:
    image: postgres:16
    environment:
      - POSTGRES_PASSWORD=secret
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - app-net

volumes:
  db-data:

networks:
  app-net:
```

```
docker compose up -d      # démarrer
docker compose logs -f    # suivre les logs
docker compose down       # arrêter et nettoyer
```

💡 Notez : `web` joint `db` par son **nom de service** (DNS interne Docker) → vos connaissances DNS s'appliquent.

4. Reverse proxy Nginx devant des conteneurs

Scénario réel : Nginx reçoit le trafic public (443) et le redirige vers les conteneurs internes.

```
server {
    listen 443 ssl;
    server_name app.exemple.com;

    ssl_certificate      /etc/letsencrypt/live/app.exemple.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/app.exemple.com/privkey.pem;

    location / {
        proxy_pass http://localhost:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location /api/ {
        proxy_pass http://localhost:4000;
    }
}
```

Architecture :

```
Internet → :443 Nginx (reverse proxy) → :3000 frontend
                                           → :4000 backend
                                           → :5432 base de données (privé)
```

5. Certificat HTTPS gratuit avec Let's Encrypt

```
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d app.exemple.com
# Renouvellement automatique testé via :
sudo certbot renew --dry-run
```

6. CI/CD : enchaîner Linux + réseau + scripts

Un pipeline CI/CD automatise build → test → déploiement. Exemple `.gitlab-ci.yml` :

```
stages:
  - build
  - test
  - deploy

build:
  stage: build
  script:
    - docker build -t monapp:$CI_COMMIT_SHORT_SHA .

test:
  stage: test
  script:
    - npm install
    - npm test

deploy:
  stage: deploy
  script:
    - ssh deploy@serveur "docker pull monapp && docker compose up -d"
  only:
    - main
```

🎯 Le déploiement utilise **SSH** + **scripts bash** + **Docker** : tout ce que vous avez appris !

7. Infrastructure as Code (notion)

Au lieu de configurer les serveurs à la main, on les décrit dans du code.

Ansible (configuration) — exemple playbook :

```
- hosts: serveurs_web
  become: yes
  tasks:
    - name: Installer nginx
      apt:
        name: nginx
        state: present
    - name: Démarrer nginx
      service:
        name: nginx
        state: started
        enabled: yes
```

```
ansible-playbook -i inventaire deploy.yml
```

Terraform (provisionnement cloud) crée des VM, réseaux, VPC... avec des fichiers `.tf`.

8. Monitoring (notion)

Surveiller pour réagir avant la panne : - **Prometheus** : collecte des métriques (CPU, RAM, requêtes). - **Grafana** : tableaux de bord visuels. - **ELK / Loki** : centralisation des logs. - **Alertmanager** : alertes (Slack, email).

Métriques système rapides (les bases) :

```
uptime; free -h; df -h; ss -s
```

9. Projet fil rouge

Objectif : déployer une application web complète, sécurisée et monitorée.

Étapes : 1. Provisionner un serveur Linux (VM ou cloud). 2. Durcir la sécurité (user sudo, SSH par clé, UFW, fail2ban). 3. Installer Docker et docker-compose. 4. Déployer une app + base de données via docker-compose. 5. Mettre un reverse proxy Nginx + HTTPS Let's Encrypt. 6. Écrire un script bash de sauvegarde planifié par cron. 7. Configurer le pare-feu (seuls 22, 80, 443 ouverts). 8. Diagnostiquer la chaîne avec `curl`, `ss`, `dig`, `journalctl`.

Exercices

1. Lancez un conteneur nginx accessible sur le port 8080 de votre machine.
2. Écrivez un Dockerfile pour une petite app et construisez l'image.
3. Créez un `docker-compose.yml` avec une app et une base PostgreSQL.
4. Configurez un reverse proxy Nginx vers un conteneur.
5. Écrivez un script de déploiement qui pull une image et redémarre via compose.
6. Listez les ports ouverts par vos conteneurs avec `ss` ou `docker ps`.

✓ Passez au [Module 12 — Travaux pratiques](#).

Module 12 — Travaux Pratiques & Exercices corrigés

 **Objectif** : consolider par la pratique. Essayez **avant** de regarder les corrigés.

TP 1 — Exploration du système

Énoncé : Collectez les informations système d'une machine Linux.

Corrigé

```
uname -a           # noyau et architecture
cat /etc/os-release # distribution
nproc              # nombre de CPU
free -h           # mémoire
df -h             # disques
ip a              # réseau
uptime            # charge et durée de fonctionnement
who               # utilisateurs connectés
```

TP 2 — Manipulation de fichiers et texte

Énoncé : À partir d'un fichier `access.log`, comptez le nombre de requêtes avec un code 404 et affichez les 5 IP les plus fréquentes.

Corrigé

```
# Nombre de 404
grep " 404 " access.log | wc -l

# Top 5 des IP (1er champ)
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -5
```

TP 3 — Utilisateurs et permissions

Énoncé : Créez un user `webadmin`, un groupe `web`, un dossier `/srv/site` appartenant à `webadmin:web` avec permissions `750`.

Corrigé

```
sudo groupadd web
sudo useradd -m -s /bin/bash -G web webadmin
sudo mkdir -p /srv/site
sudo chown -R webadmin:web /srv/site
sudo chmod 750 /srv/site
ls -ld /srv/site          # vérification
```

TP 4 — Script de monitoring

Énoncé : Écrivez un script qui affiche une alerte si l'utilisation disque de `/` dépasse 80 %.

Corrigé

```
#!/bin/bash
set -euo pipefail

SEUIL=80
USAGE=$(df / | awk 'NR==2 {print $5}' | tr -d '%')

if [ "$USAGE" -ge "$SEUIL" ]; then
    echo "[ALERTE] Disque / à ${USAGE}% (seuil ${SEUIL}%)" >&2
    exit 1
else
    echo "[OK] Disque / à ${USAGE}%"
fi
```

TP 5 — Service systemd

Énoncé : Créez un service systemd qui lance un script `/opt/hello/run.sh` au démarrage et le redémarre s'il plante.

Corrigé

```
# 1. Le script
sudo mkdir -p /opt/hello
echo -e '#!/bin/bash\nwhile true; do echo "vivant $(date)"; sleep 10; done' \
| sudo tee /opt/hello/run.sh
sudo chmod +x /opt/hello/run.sh

# 2. Le service : /etc/systemd/system/hello.service
```

```
[Unit]
Description=Service Hello
After=network.target

[Service]
ExecStart=/opt/hello/run.sh
Restart=always

[Install]
WantedBy=multi-user.target
```

```
# 3. Activer
sudo systemctl daemon-reload
sudo systemctl enable --now hello
sudo systemctl status hello
journalctl -u hello -f
```

TP 6 — Subnetting

Énoncé : On vous donne **172.16.0.0/16**. Découpez-le pour obtenir au moins 8 sous-réseaux. Donnez le nouveau masque et les 3 premiers sous-réseaux.

Corrigé

- 8 sous-réseaux = 2^3 → emprunter 3 bits → **/16 + 3 = /19**
- Masque **/19 = 255.255.224.0**
- Incrément du 3e octet : $256 - 224 = 32$

Sous-réseau	Plage
172.16.0.0/19	172.16.0.1 - 172.16.31.254
172.16.32.0/19	172.16.32.1 - 172.16.63.254
172.16.64.0/19	172.16.64.1 - 172.16.95.254

Chaque sous-réseau : $2^{(32-19)} - 2 = \mathbf{8190}$ hôtes.

TP 7 — Diagnostic réseau

Énoncé : Un site `monapp.local` ne répond pas. Décrivez et exécutez la démarche de diagnostic complète.

Corrigé

```
# 1. Résolution DNS
dig +short monapp.local
nslookup monapp.local
cat /etc/hosts                # résolution locale ?

# 2. Connectivité réseau
ping -c 3 monapp.local
ping -c 3 <IP>                 # contourner le DNS

# 3. Port ouvert ?
nc -zv monapp.local 443
ss -tulpn | grep 443          # côté serveur

# 4. Application répond ?
curl -Iv https://monapp.local

# 5. Logs côté serveur
sudo journalctl -u nginx -n 50
sudo tail -f /var/log/nginx/error.log
```

Conclusion type : si DNS échoue → corriger `/etc/hosts` ou le DNS ; si port fermé → pare-feu ou service arrêté ; si 502 → backend down.

TP 8 — Pare-feu

Énoncé : Sécurisez un serveur web avec UFW : seuls SSH (depuis votre réseau local uniquement), HTTP et HTTPS doivent être accessibles.

Corrigé

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow from 192.168.1.0/24 to any port 22 proto tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status verbose
```

TP 9 — Déploiement Docker complet

Énoncé : Déployez une app web + Redis via docker-compose, exposée sur le port 8080.

Corrigé

`docker-compose.yml` :

```
version: "3.8"
services:
  app:
    image: nginx:alpine
    ports:
      - "8080:80"
    depends_on:
      - cache
  cache:
    image: redis:alpine
```

```
docker compose up -d
curl -I http://localhost:8080      # vérifier
docker compose ps
docker compose logs -f
```

TP 10 — Automatisation par cron

Énoncé : Planifiez une sauvegarde quotidienne d'un dossier à 2h du matin, avec rotation (garder 7 jours).

Corrigé

Script `/opt/scripts/backup.sh` :

```
#!/bin/bash
set -euo pipefail
SRC="/srv/site"
DEST="/backups"
mkdir -p "$DEST"
tar -czf "$DEST/site_$(date +%F).tar.gz" "$SRC"
find "$DEST" -name "site_*.tar.gz" -mtime +7 -delete
```

```
chmod +x /opt/scripts/backup.sh
crontab -e
# ajouter :
0 2 * * * /opt/scripts/backup.sh >> /var/log/backup.log 2>&1
```

Évaluation finale (auto-test)

Cochez ce que vous savez faire sans aide :

- ☐ Naviguer et manipuler des fichiers en ligne de commande
- ☐ Gérer utilisateurs, groupes et permissions
- ☐ Gérer des services avec systemd et lire les logs
- ☐ Installer des paquets et gérer le stockage
- ☐ Écrire un script bash robuste avec conditions et boucles
- ☐ Expliquer les modèles OSI / TCP/IP
- ☐ Calculer un sous-réseau (subnetting) en CIDR
- ☐ Utiliser DNS, HTTP, SSH, curl
- ☐ Diagnostiquer un problème réseau couche par couche
- ☐ Configurer un pare-feu et sécuriser SSH
- ☐ Déployer une app conteneurisée avec reverse proxy

Pour aller plus loin

- **Certifications** : LPIC-1, RHCSA, CompTIA Linux+, CKA (Kubernetes).
- **Pratique** : montez un homelab (VirtualBox/Proxmox), KillerCoda, Katacoda-like.
- **Lectures** : « The Linux Command Line » (W. Shotts), documentation Docker/Kubernetes.
- **Suite logique** : Git avancé, Kubernetes, Terraform, Ansible, observabilité.



Félicitations ! Vous avez les fondations Linux & Réseau d'un DevOps.



Retour au [sommaire](#).